

Ques 1

```
In [1]: import pandas as pd

# Sample Dataset
data = {
    'Product ID': ['P001', 'P002', 'P003'],
    'Sales Amount': [150, 200, 120]
}

df = pd.DataFrame(data)

# Analysis
highest_selling = df.loc[df['Sales Amount'].idxmax()]

print("=== Sales Report ===")
print(f"Highest Selling Product: {highest_selling['Product ID']}")
print(f"Sales Amount: ${highest_selling['Sales Amount']}")
print("\nFull Sales Data:")
print(df)
```

```
=== Sales Report ===
Highest Selling Product: P002
Sales Amount: $200
```

```
Full Sales Data:
  Product ID  Sales Amount
0       P001           150
1       P002           200
2       P003           120
```

Summary:

This program loads sales data into a pandas DataFrame, identifies the product with maximum sales, and generates a clear report. Easily extendable for large CSV files using `pd.read_csv()`.

Ques 2

```
In [2]: import math

class Shape:
    def calculate_area(self):
        pass # To be overridden

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def calculate_area(self):
        return math.pi * self.radius ** 2

class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def calculate_area(self):
        return self.length * self.width

# Example Usage
circle = Circle(5)
rectangle = Rectangle(4, 6)

print(f"Circle Area: {circle.calculate_area():.2f}")
print(f"Rectangle Area: {rectangle.calculate_area():.2f}")
```

Circle Area: 78.54
Rectangle Area: 24.00

Summary:

Demonstrates Inheritance and Method Overriding in OOP.

Base class Shape provides structure. Circle uses πr^2 , Rectangle uses $\text{length} \times \text{width}$.

Polymorphism allows different shapes to use the same method name.

Ques 3

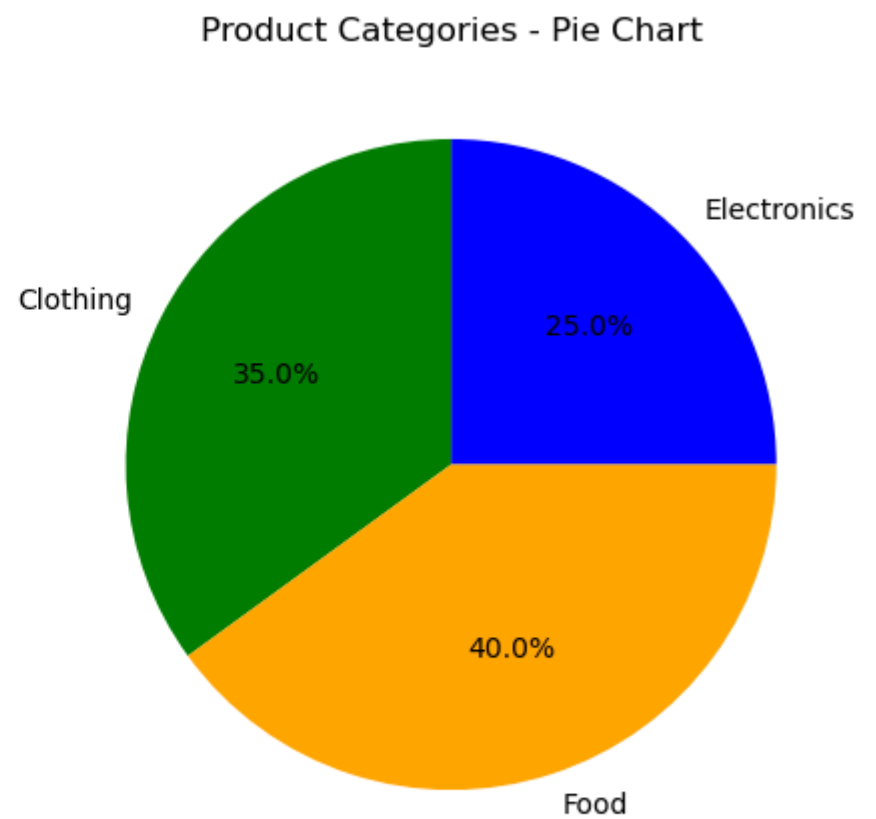
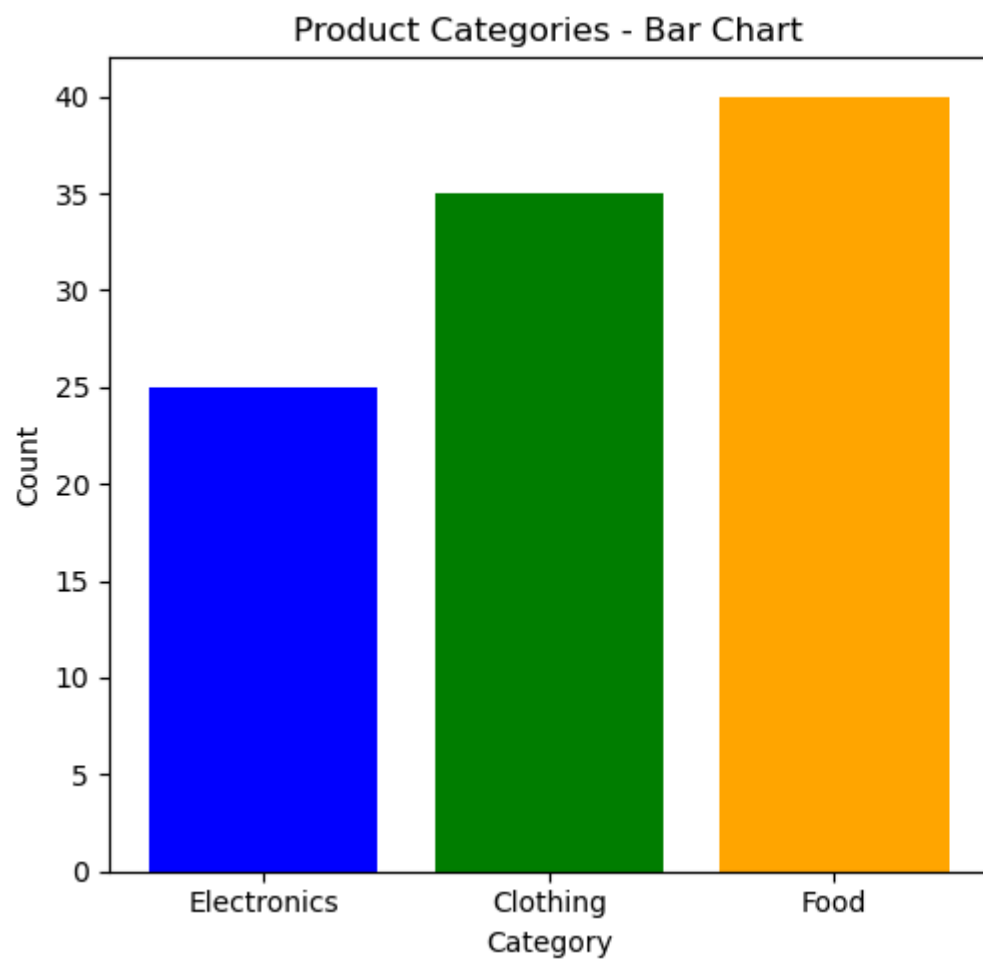
In [3]: `import matplotlib.pyplot as plt`

```
# Sample Dataset
categories = ['Electronics', 'Clothing', 'Food']
counts = [25, 35, 40]

# Bar Chart
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.bar(categories, counts, color=['blue', 'green', 'orange'])
plt.title('Product Categories - Bar Chart')
plt.xlabel('Category')
plt.ylabel('Count')

# Pie Chart
plt.subplot(1, 2, 2)
plt.pie(counts, labels=categories, autopct='%1.1f%%', colors=['blue', 'green', 'orange'])
plt.title('Product Categories - Pie Chart')

plt.tight_layout()
plt.show()
```



Summary:

Creates both bar chart and pie chart using Matplotlib to visualize category distribution.

Bar chart is good for comparison, pie chart shows proportional distribution.

Ques 4

```
In [5]: class Animal:
def speak(self):
return "Some sound"
```

```
class Dog(Animal):
    def speak(self):
        return "Woof"

class Cat(Animal):
    def speak(self):
        return "Meow"

# Example Usage
dog = Dog()
cat = Cat()

print(f"Dog says: {dog.speak()}")
print(f"Cat says: {cat.speak()}")
```

Dog says: Woof

Cat says: Meow

Summary:

Demonstrates Inheritance and Method Overriding (Polymorphism).

Child classes Dog and Cat provide specific implementations of the speak() method.

Ques 5

```
In [7]: import random

students = ["Alice", "Bob", "Eve", "David"]

# Generate random order
random_order = students.copy()
random.shuffle(random_order)

print("Random Presentation Order:")
for i, student in enumerate(random_order, 1):
    print(f"{i}. {student}")
```

Random Presentation Order:

1. David
2. Alice
3. Eve
4. Bob

Summary:

Reads list of students and shuffles it randomly using `random.shuffle()`.

Useful for fair randomization of presentation or speaking order.

Ques 6

```
In [8]: list1 = [1, 2, 3, 4, 5]
list2 = [3, 4, 5, 6, 7]

# Method 1: Using sets (most efficient)
common = list(set(list1) & set(list2))

# Method 2: List comprehension
# common = [x for x in list1 if x in list2]

print("Common elements:", common)
```

Common elements: [3, 4, 5]

Summary:

Finds intersection between two lists efficiently using set operations.

Time complexity improves from $O(n \times m)$ to $O(n+m)$ using sets.

Ques 7

```
In [10]: dict1 = {"A": 1, "B": 2, "C": 3}
dict2 = {"B": 4, "C": 5, "D": 6}
```

```
common_keys = list(set(dict1.keys()) & set(dict2.keys()))  
  
print("Common keys:", common_keys)
```

Common keys: ['B', 'C']

Summary:

Extracts common keys from two dictionaries using set intersection on keys.

Very useful in data comparison and merging operations.

In []: